

```
In [ ]: #Linear Search
class solution:
    def LinearSearch(self,nums:list[str],target:int)->int:
        for i in range(len(nums)):
            if nums[i]==target:
                return i
        return -1
obj=solution()
r=obj.LinearSearch([1,2,3,4,8,7,6],4)
print(f'Result is:{r}')
```

Result is:3

```
In [5]: #Binary search
class solution:
    def BinarySearch(self,nums:list[int],target:int)->int:
        i=0
        n=len(nums)
        j=n-1

        while i<j:
            mid=(i+j)//2

            if nums[mid]==target:
                return mid
            if nums[mid]>target:
                i=mid+1
            else:
                j=mid-1
        return -1
obj=solution()
r=obj.BinarySearch([1,2,3,4,8,7,6],4)
print(f'Result is:{r}')
```

Result is:3

```
In [ ]: #Lower Bond
class solution:
    def LowerBound(self,nums:list[int],target:int)->int:
        l=0
        n=len(nums)
        r=n-1
        ans=n

        while l<r:
            mid=(l+r)//2

            if nums[mid]>=target:
                ans=mid
                r=mid-1
            else:
                l=mid+1
        return ans

#Upper Bound
class solution:
    def LowerBound(self,nums:list[int],target:int)->int:
        l=0
        n=len(nums)
```

```

r=n-1
ans=n

while l<r:
    mid=(l+r)//2

    if nums[mid]>target:
        ans=mid
        r=mid-1
    else:
        l=mid+1
return ans

```

```

In [6]: #Search insert position
class solution:
    def LowerBound(self,nums:list[int],target:int)->int:
        n=len(nums)
        l=0
        r=n-1
        ans=n

        while l<r:
            mid=(l+r)//2

            if nums[mid]>=target:
                ans=mid
                r=mid-1
            else:
                l=mid+1
        return ans
    def SearchInsertPosition(self,nums:list[int],target:int)->int:
        return self.LowerBound(nums,target)
obj=solution()
r=obj.SearchInsertPosition([1,3,5,6],5)
print(f'Result is:{r}')

```

Result is:2

```

In [2]: #Find first and last last position of element in a sorted array
class solution:
    def LowerBound(self,nums:list[int],target:int)->int:
        l=0
        n=len(nums)
        r=n-1
        ans=n

        while l<=r:
            mid=(l+r)//2

            if nums[mid]>=target:
                ans=mid
                r=mid-1
            else:
                l=mid+1
        return ans

    #Upper Bound

    def UpperBound(self,nums:list[int],target:int)->int:
        l=0

```

```

n=len(nums)
r=n-1
ans=n

while l<=r:
    mid=(l+r)//2

    if nums[mid]>target:
        ans=mid
        r=mid-1
    else:
        l=mid+1
return ans

def FindingPosition(self,nums:list[int],target:int)->list[int]:
    lb=self.LowerBound(nums,target)
    ub=self.UpperBound(nums,target)

    if lb==ub:
        return [-1,-1]
    else:
        return [lb,ub-1]
nums=[5,7,7,8,8,10]
target=8
obj=solution()
r3=obj.FindingPosition(nums,target)
print(f'Result is:{r3}')

```

Result is:[3, 4]

In [3]: *#Search inside rotated sorted array*

```

class solution:
    def Search(self,nums:list[int],target:int)->int:
        n=len(nums)-1
        l=0
        r=n-1

        while l<=r:
            mid=(l+r)//2

            if nums[mid]==target:
                return mid

            if nums[l]<nums[mid]:
                if nums[l]<target<nums[mid]:
                    r=mid-1
                else:
                    l=mid+1
            else:
                if nums[r]<target<nums[mid]:
                    r=mid-1
                else:
                    l=mid+1

        return 1
obj=solution()
ans=obj.Search([4,5,6,7,0,1,2],0)
print(f'Answer is:{ans}')

```

Answer is:4

```
In [ ]: #sort array by parity
class solution:
    def SortByParity(self,nums:list[int])>->list[int]:
        start=0
        for i in range(len(nums)):
            if nums[i]%2==0:
                temp=nums[i]
                nums[i]=nums[start]
                nums[start]=temp
                start+=1
        return nums
obj=solution()
r=obj.SortByParity([3,5,7,2,4,1])
print(f'Result is:{r}')
```

Result is:[2, 4, 7, 3, 5, 1]

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](\"https://aka.ms/vscodeJupyterKernelCrash\") for more info.

View Jupyter [log](\"command:jupyter.viewOutput\") for further details.